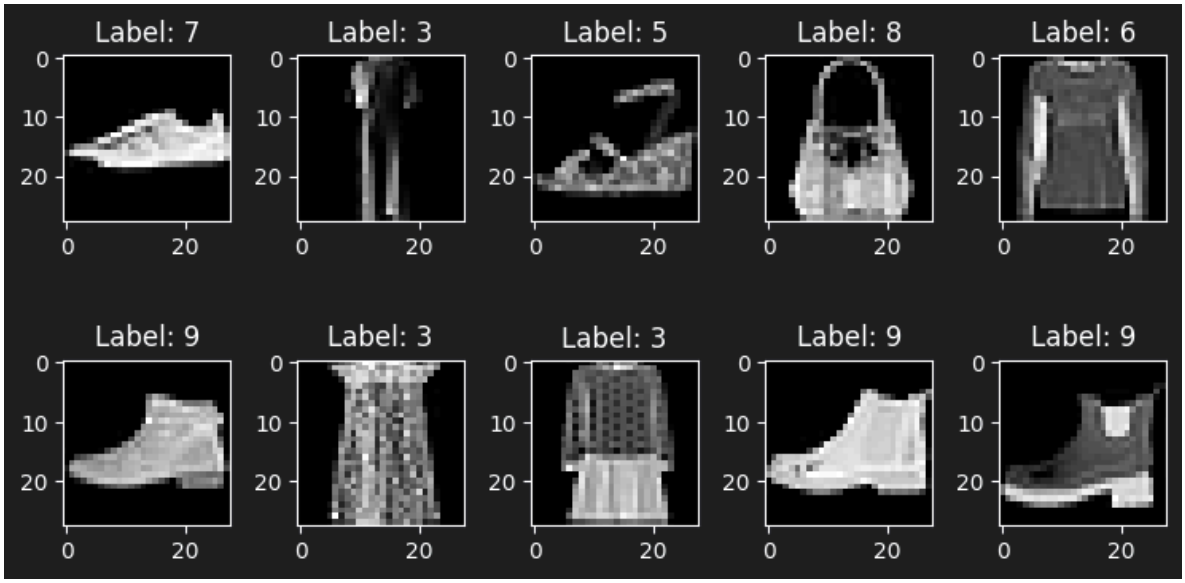
Challenge 3 on FashionMNIST

1. Data Exploration and Preprocessing



The first step of my project involved exploring the FashionMNIST dataset. After visualizing a sample of the images to understand the visual complexity of the clothing items, I split the data into training, validation, and testing sets.

To improve my model's ability to generalize and mitigate the slight overfitting I observed in early runs, I implemented Data Augmentation. Specifically, I introduced a `RandomHorizontalFlip` (with a 50% probability) and a `RandomRotation` (up to 10 degrees) to artificially increase the diversity of the training images. A critical aspect of my implementation was preventing data leakage: I had to ensure that the validation set remained completely untouched by these random transformations. To do this, I instantiated the `FashionMNIST` dataset twice—once with the full augmentation pipeline and once with only the base transformations (`ToTensor` and `Normalize`). Using PyTorch's `Subset`, I then securely assigned the training indices to the augmented dataset and the validation indices to the "clean" dataset. This guaranteed a strict and reliable evaluation of the model during training.

2. Architecture and Hyperparameter Tuning

For this project, I experimented with two main types of architectures: Fully Connected (FC) Neural Networks and Convolutional Neural Networks (CNN), tuning the number of hidden layers (up to 3), the optimizer, and the hyperparameters.

Why depth and convolutions improve results:

The baseline FC model with only 1 Hidden Layer (HL) severely underperformed. A single layer simply lacks the representational power needed to capture the complex, non-linear visual patterns of this dataset. Furthermore, FC networks require flattening the 2D image into a 1D array, completely destroying the spatial relationships between pixels. Upgrading to a deeper 3HL architecture provided a massive jump in performance because more layers increase the network's capacity to learn.

However, Convolutional Neural Networks proved to be fundamentally superior. Instead of flattening the image, convolutions slide filters over the 2D space, preserving local spatial context. Expanding the architecture to 3 Convolutional layers expands the network's "receptive field": early layers detect simple edges, while deeper layers combine these into complex textures and abstract shapes (like distinguishing a shirt collar from a coat lapel).

Hyperparameter Choices: Dropout, Batch Normalization, and Schedulers:

To support these deep networks and prevent overfitting, I integrated several crucial techniques:

- **Batch Normalization:** I added BN layers to standardize the inputs of each hidden layer for every mini-batch. This choice significantly stabilizes the learning process by reducing the internal covariate shift. It allows the network to converge faster and more reliably, even with higher learning rates.
- **Dropout (and Dropout2d):** Deep dense layers contain the vast majority of the network's parameters, carrying a massive risk of overfitting. I introduced Dropout to randomly zero out neurons during training, which prevents neurons from co-adapting and forces the network to learn robust, redundant features. For the CNNs, I used `Dropout2d` to drop entire feature maps, promoting spatial independence.
- **Learning Rate Scheduler:** I utilized a scheduler to dynamically adjust the learning rate. Starting with a higher learning rate allows the optimizer to quickly traverse the loss landscape and escape bad local minima. Over time, the scheduler reduces the learning rate, allowing the model to gently settle into the flattest, most optimal minimum without oscillating or overshooting.
- **Early Stopping:** To avoid unnecessary computation, I implemented a custom `EarlyStopping` class (with `patience=5` and `min_delta=0.005`) that monitors validation loss and halts training the moment generalization stops improving.

The Optimizers: Adam vs AdamW vs SGD:

I rigorously tested three optimizers. Standard Adam adapts the learning rate per parameter, offering fast and stable convergence. However, its adaptive gradients inadvertently "eat" the L2 penalty, making weight decay less effective for generalization. AdamW solves this by decoupling the weight decay from the gradient update, applying it directly to the weights. This restores true regularization while keeping Adam's speed. Lastly, SGD (with Nesterov momentum) takes a slower, steadier path. While it takes longer to train, it is mathematically known to navigate towards flatter, wider minima in the loss landscape, which generalize phenomenally well on unseen data.

3. Results Summary

Rank	Model	Optimizer	Best Val Loss	Best Val Acc	Test Acc
1	CNN 3 hidden layers	SGD	0.6670	92.82%	92.49%
2	CNN 3 hidden layers	Adam	0.6810	92.33%	91.82%
3	CNN 3 hidden layers	AdamW	0.6858	92.26%	91.82%
4	CNN 1 hidden layer	Adam	0.7306	90.08%	89.85%
5	FC 3 hidden layers	AdamW	0.7339	89.35%	88.54%
...	(full table in notebook)	...	...	...	...

The final evaluation on the test set provides clear insights into how architectural depth and optimization strategies affect the model.

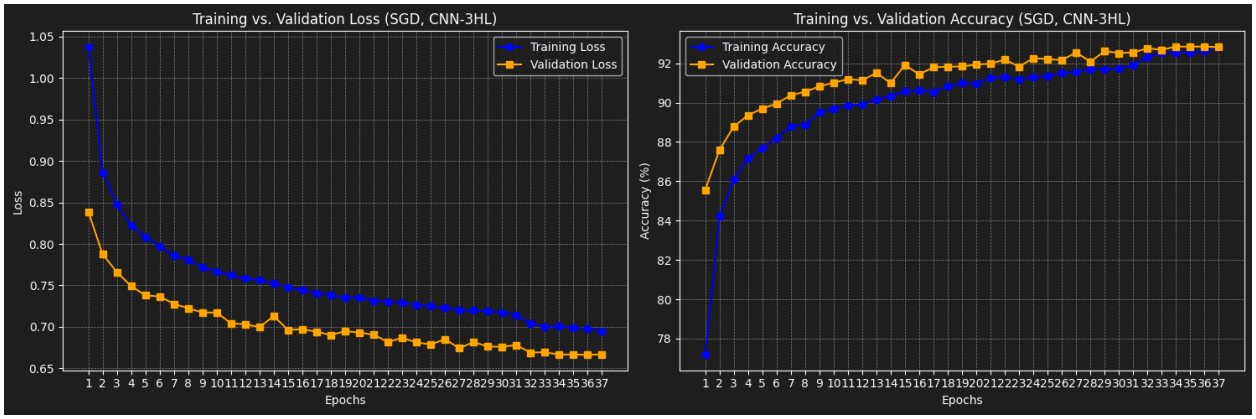
Analyzing architectural capacity, the baseline FC 1HL (Adam) achieved only 82.85% accuracy. Upgrading to a FC 3HL (Adam) boosted this to 86.99%. However, the CNN architectures completely outclassed the FC models: even the simple CNN 1HL (Adam) hit 89.85%, proving

that extracting 2D spatial features is better suited for FashionMNIST. The deep CNN 3HL models easily pushed past 91%.

Comparing optimizers validated my optimization hypotheses:

- In the FC architecture, AdamW effectively decoupled the weight decay and successfully regularized the network, matching the strong generalization of SGD (~88.5%) but achieving the absolute lowest test loss (0.7460), whereas standard Adam struggled.
- In the deep CNN architecture, standard Adam and AdamW achieved an identical, excellent accuracy of 91.82%. This suggests that my aggressive spatial regularization (Data Augmentation + Dropout2d) already did the heavy lifting in preventing overfitting. Yet, the undisputed winner was SGD. By taking a steady path, it secured the lowest test loss (0.6745) and the highest overall accuracy (92.49%), confirming that a well-tuned SGD remains the gold standard for maximizing generalization.

4. Learning Curves, Confusion Matrix, and Metric Evaluation

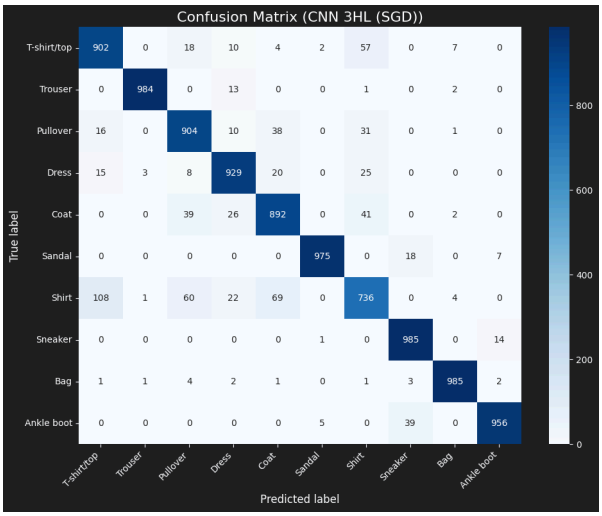


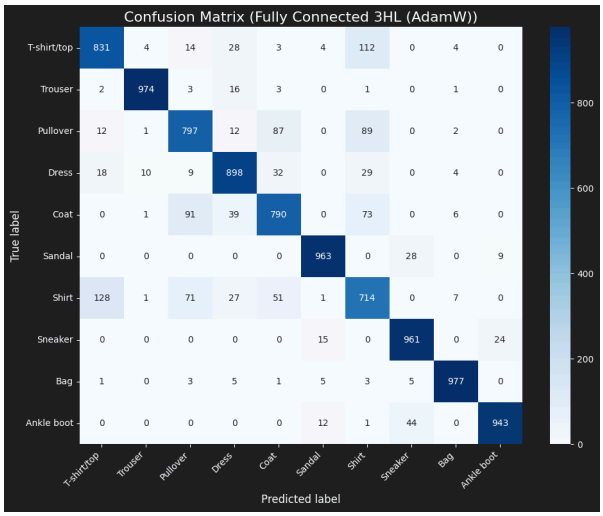
The plotted learning curves illustrate smooth convergence. The gap between training and validation loss remains controlled, and the early stopping mechanism clearly cuts off the training exactly when the validation loss begins to plateau, preventing overfitting.

Is Accuracy a good metric here?

In this specific project, accuracy is an excellent global metric. The FashionMNIST dataset is perfectly balanced: each category contains the exact same number of samples (6,000 for training and 1,000 for testing). Because there is no class imbalance, we do not suffer from the "accuracy paradox"—a situation where a model achieves a high score simply by always guessing the majority class without actually learning. Here, an accuracy of 92% genuinely means the model correctly identifies 92 out of 100 items.

Confusion Matrix Analysis:





However, I must note a key limitation: while accuracy gives me the overall "grade", it does not tell me *where* the model is failing. This is exactly why I generated and analyzed the Confusion Matrix. This tool allowed me to look under the hood and discover that the remaining errors were not random. They were heavily clustered around visually ambiguous upper-body garments—specifically Shirts, T-shirts, Pullovers, and Coats. The model easily distinguishes a sneaker from a bag, but it understandably struggles with the fine nuances between long-sleeved tops, which aligns perfectly with human visual ambiguity.